Experiment No: 03

Title: Implementation of different types of functions with suitable examples

Topics to be covered:

- Number Functions (`ABS`, `ROUND`, `MOD`, `CEIL`, `FLOOR`)
- Aggregate Functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`)
- Character/String Functions (`UPPER`, `LOWER`, `LENGTH`, `SUBSTR`, `INSTR`)
- Conversion Functions (`TO_CHAR`, `TO_DATE`, `CAST`, `CONVERT`)
- Date Functions (`SYSDATE`, `CURRENT_DATE`, `MONTHS_BETWEEN`, `ADD_MONTHS`, `NEXT_DAY`)

Types of SQL Functions with Examples

SQL provides a variety of functions that can be used to manipulate data, perform calculations, and handle different types of queries. Below are different categories of functions in SQL, with examples and explanations.

1. Number Functions

Number functions in SQL allow you to perform mathematical operations on numeric data types.

- **Examples:**
  - `ABS(number)`: Returns the absolute value of a number.
  - `CEIL(number)`: Returns the smallest integer greater than or equal to the specified number.
  - `FLOOR(number)`: Returns the largest integer less than or equal to the specified number.
  - `ROUND(number, decimals)`: Rounds a number to a specified number of decimal places.
  - `MOD(dividend, divisor)`: Returns the remainder when one number is divided by another.

Example:

```
SELECT ABS(-10) AS Absolute_Value,
       CEIL(4.3) AS Ceil_Value,
       FLOOR(4.7) AS Floor_Value,
       ROUND(15.789, 2) AS Rounded_Value,
       MOD(10, 3) AS Modulus_Value;
```

2. Aggregate Functions

Aggregate functions in SQL are used to perform calculations on multiple rows of a dataset and return a single value.

- **Examples:**
  - `COUNT(column_name)`: Returns the number of rows that match a specified condition.
  - `SUM(column_name)`: Returns the sum of the values in a specified column.
  - `AVG(column_name)`: Returns the average value of a numeric column.

- `MIN(column_name)`: Returns the smallest value in a specified column.
- `MAX(column_name)`: Returns the largest value in a specified column.

Example:

```
SELECT COUNT(*) AS Total_Employees,
       SUM(salary) AS Total_Salary,
       AVG(salary) AS Average_Salary,
       MIN(salary) AS Minimum_Salary,
       MAX(salary) AS Maximum_Salary
FROM employees;
```

3. Character Functions / String Matching

Character functions in SQL are used to manipulate string data types.

- **Examples:**
  - `CONCAT(string1, string2)`: Concatenates two or more strings.
  - `LENGTH(string)`: Returns the length of a string.
  - `UPPER(string)`: Converts a string to uppercase.
  - `LOWER(string)`: Converts a string to lowercase.
  - `TRIM(string)`: Removes leading and trailing spaces from a string.
  - `SUBSTRING(string, start_position, length)`: Extracts a substring from a string.

Example:

```
SELECT CONCAT('Hello ', 'World!') AS Full_Name,
       LENGTH('SQL') AS String_Length,
       UPPER('sql') AS Uppercase,
       LOWER('SQL') AS Lowercase,
       TRIM('  SQL  ') AS Trimmed_String,
       SUBSTRING('Programming', 1, 5) AS Substring;
```

4. Conversion Functions

Conversion functions in SQL allow you to convert data from one type to another.

- **Examples:**
  - `CAST(expression AS target_data_type)`: Converts an expression to the specified data type.
  - `CONVERT(target_data_type, expression)`: Similar to `CAST`, but syntax differs between SQL implementations.

Example:

```
SELECT CAST(123.45 AS INT) AS Converted_Int; //MySQL (XAMPP)

SELECT CONVERT(VARCHAR, 2025) AS Converted_Varchar; //SQL Server
```

5. Date Functions

Date functions in SQL are used to manipulate date and time values.

- **Examples:**
  - `CURRENT_DATE`: Returns the current date.
  - `CURRENT_TIME`: Returns the current time.
  - `DATE_ADD(date, INTERVAL value unit)`: Adds a specified time interval to a date.
  - `DATE_SUB(date, INTERVAL value unit)`: Subtracts a specified time interval from a date.
  - `DATEDIFF(date1, date2)`: Returns the difference in days between two dates.

  Example:

```
SELECT CURRENT_DATE AS Today,
       CURRENT_TIME AS Current_Times,
       DATE_ADD('2025-02-17', INTERVAL 5 DAY) AS Date_After_5_Days,
       DATE_SUB('2025-02-17',      INTERVAL      3      MONTH)      AS
Date_Before_3_Months,
       DATEDIFF('2025-02-17', '2025-01-01') AS Days_Difference;
```

Example Table: Employees

employee_id	first_name	last_name	salary	hire_date
1	John	Doe	60000	2020-01-15
2	Jane	Smith	70000	2019-03-23
3	Tom	Brown	75000	2021-07-01
4	Alice	Green	85000	2018-11-05
5	Bob	White	50000	2022-09-12

Example Questions and Answers

1. **Question 1:**
   - Find the total number of employees and the average salary of all employees.

   Solution:

```
SELECT COUNT(*) AS Total_Employees,
       AVG(salary) AS Average_Salary
FROM employees;
```

2. **Question 2:**
   - What is the highest and lowest salary among all employees?

   Solution:

```
SELECT MAX(salary) AS Highest_Salary,
       MIN(salary) AS Lowest_Salary
FROM employees;
```

3. **Question 3:**
   - Convert the salary of an employee from `FLOAT` to `INT` for employee with `employee_id = 1`.

Solution:

```
SELECT CAST(salary AS INT) AS Converted_Salary
FROM employees
WHERE employee_id = 1;
```

4. **Question 4:**
   - What is the length of the first name of employee with `employee_id = 2`?

Solution:

```
SELECT LENGTH(first_name) AS First_Name_Length
FROM employees
WHERE employee_id = 2;
```

5. **Question 5:**
   - Find the number of days between today and the hire date of employee with `employee_id = 3`.

Solution:

```
SELECT DATEDIFF(CURRENT_DATE, hire_date) AS Days_Worked
FROM employees
WHERE employee_id = 3;
```

6. **Question 6:**
   - Find the full name of all employees by concatenating their first and last names.

Solution:

```
SELECT CONCAT(first_name, ' ', last_name) AS Full_Name
FROM employees;
```

7. **Question 7:**
   - What is the date 3 months from the hire date of employee `employee_id = 4`?

Solution:

```
SELECT DATE_ADD(hire_date, INTERVAL 3 MONTH) AS Date_After_3_Months
FROM employees
WHERE employee_id = 4;
```

SQL LIKE Operator: try yourself form

https://www.w3schools.com/sql/sql_like.asp

LAB Task-1:

Table: Employees

employee_id	first_name	last_name	salary	hire_date	department	date_of_birth	gender	city
1	John	Doe	60000	2020-01-15	IT	1985-08-12	M	New York
2	Jane	Smith	70000	2019-03-23	HR	1990-05-22	F	Chicago
3	Tom	Brown	75000	2021-07-01	Sales	1988-12-04	M	Boston
4	Alice	Green	85000	2018-11-05	IT	1987-07-19	F	Miami
5	Bob	White	50000	2022-09-12	Marketing	1995-03-30	M	Dallas
6	Carol	Harris	65000	2020-10-25	HR	1983-11-03	F	New York
7	Kevin	Lewis	72000	2019-02-19	Sales	1992-06-07	M	Chicago
8	Rachel	Adams	68000	2017-08-14	Marketing	1991-01-13	F	Boston
9	Michael	Scott	95000	2016-03-15	Sales	1980-10-10	M	Dallas
10	Sophie	Clark	71000	2019-04-10	IT	1989-02-20	F	Miami

1. Find the absolute value of each employee's salary (even though salaries are positive).
2. What is the total salary of all employees in the `Sales` department?
3. Extract the first 3 letters of each employee's first name.
4. Find the number of days between today and the hire date of each employee.
5. Convert the `salary` from numeric to `VARCHAR` data type for each employee
6. What is the average salary of employees in the `IT` department?
7. Convert all employee last names to uppercase.
8. Round each employee's salary to the nearest thousand.
9. Find the hire date of each employee 6 months from now.
10. Concatenate the first and last name of each employee with a space in between.

LAB Task-2:

Example Table: Employees

```
CREATE TABLE Employees (
    emp_id INT,
    first_name VARCHAR(20),
    last_name VARCHAR(20),
    salary DECIMAL(10,2),
    hire_date DATE,
    dept VARCHAR(20),
    bonus_percent DECIMAL(5,2)
);
```

Sample Data

emp_id	first_name	last_name	salary	hire_date	dept	bonus_percent
1	John	Doe	60000.00	2020-01-15	IT	10.5
2	Jane	Smith	75000.50	2019-03-23	HR	12.0
3	Alice	Green	82000.75	2021-07-10	Sales	15.75
4	Tom	Brown	50000.00	2022-05-01	Marketing	9.0
5	Bob	White	90000.00	2018-11-30	IT	11.25

1. Display the absolute value of bonus_percent for all employees.
2. Round the salary to the nearest thousand for each employee.
3. Calculate the average salary of all employees.
4. Find the highest and lowest salary.
5. Display the full name (first and last name combined) in uppercase.
6. Show employees whose first name contains the letter ‘o’.
7. Convert the salary to a character data type.
8. Convert the hire_date to 'Month YYYY' format.
9. Find how many months each employee has worked in the company.
10. Show the next Monday after each employee's hire date.

Teacher Notes:

- Use MySQL, Oracle, or PostgreSQL depending on your lab setup.
- For Oracle: `TO_CHAR, TO_DATE, MONTHS_BETWEEN, NEXT_DAY work directly.`
- In MySQL:
  - `Use DATE_FORMAT() instead of TO_CHAR.`
  - `Use TIMESTAMPDIFF(MONTH, hire_date, CURDATE()) instead of MONTHS_BETWEEN.`

Lab Task-1 Solution:

```
SELECT employee_id, first_name, last_name, ABS(salary) AS Absolute_Salary
FROM employees;

SELECT SUM(salary) AS Total_Salary_Sales
FROM employees
WHERE department = 'Sales';

SELECT employee_id, first_name, SUBSTRING(first_name, 1, 3) AS
First_3_Letters
FROM employees;

SELECT employee_id, first_name, last_name, DATEDIFF(CURRENT_DATE,
hire_date) AS Days_Employed
FROM employees;

SELECT employee_id, first_name, last_name, CONVERT(VARCHAR, salary) AS
Converted_Salary
FROM employees;

SELECT AVG(salary) AS Average_IT_Salary
FROM employees
WHERE department = 'IT';

SELECT employee_id, first_name, UPPER(last_name) AS Uppercase_Last_Name
FROM employees;

SELECT employee_id, first_name, last_name, ROUND(salary, -3) AS
Rounded_Salary
FROM employees;

SELECT employee_id, first_name, last_name, DATE_ADD(hire_date, INTERVAL 6
MONTH) AS Hire_Date_6_Months_Later
FROM employees;

SELECT employee_id, CONCAT(first_name, ' ', last_name) AS Full_Name
FROM employees;
```